SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

ERROR ANALYSIS TASK

Name of the Student	P LIKITH REDDY
Reg. No	192425134

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I P LIKITH REDDY (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

QUESTIONS: **Total Marks:25**

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.

Answer :

The conceptual error is that the likelihood is written as the sum of probabilities instead of the product of probabilities of all independent observations. In MLE, the likelihood represents the joint probability of the entire dataset.

b) Explain how this mistake affects parameter estimation.

Answer :

Using the sum instead of the product does not correctly represent the probability of observing the complete dataset. As a result, the estimated parameters do not maximize the true likelihood, leading to inaccurate parameter values and poor classification performance, even with a large amount of data.

c) Suggest the correct mathematical formulation and reasoning.

Answer :

The correct likelihood function for independent observations is:

$$L(\theta) = \prod_{i=1}^{n} P\left(x_i \mid \theta\right)$$

For easier computation, the **log-likelihood** is used:

$$\log L\left(\theta\right) = \sum_{i=1}^{n} \log P\left(x_i \mid \theta\right)$$

d) How would using log-likelihood fix computational issues?

Answer :

Using the log-likelihood converts the product of many probabilities into a sum of logarithms:

$$\log L\left(\theta\right) = \sum_{i=1}^{n} \log P\left(x_i \mid \theta\right)$$

This avoids numerical underflow caused by multiplying many very small probabilities, makes calculations more stable, and simplifies optimization by turning multiplication into addition. As a result, parameter estimation becomes more efficient and accurate.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

Answer :

- Incorrect weight update formula.
- Bias term is missing or updated incorrectly.

- Wrong prediction or activation function.
- Incorrect learning rate value.
- Errors in feature preprocessing or data handling.
- Stopping condition implemented incorrectly.

b) Analyze how incorrect learning rate or weight updates affect convergence.
Answer :
If the learning rate is too high, the weights may overshoot the optimal values and fail to converge. If it is too low, learning becomes very slow. Incorrect weight updates move the decision boundary in the wrong direction, preventing the Perceptron from finding the correct separating hyperplane.
c) Identify dataset-related issues that may cause this problem.
Answer :

- Data may not actually be linearly separable.
- Incorrect or noisy class labels.
- Features are not properly scaled or normalized.
- Presence of duplicate or inconsistent training samples.
- Missing or incorrect data values.

d) Propose modifications to ensure convergence.
Answer :

- Use the correct Perceptron weight update rule.
- Choose a suitable learning rate (e.g., 0.01 or 0.1).
- Include and update the bias term correctly.
- Normalize or standardize input features.
- Verify that the dataset is linearly separable and free from labeling errors.
- Continue training until no misclassified samples remain or the maximum number of iterations is reached.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.
Tasks:
a) Identify possible mathematical or coding errors in gradient computation.
Answer :

b) Explain the role of activation functions in gradient flow.
Answer :

c) Analyze how vanishing or exploding gradients affect training.
Answer :

d) Suggest techniques to fix the issue (e.g., normalization, initialization).
Answer :

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.
Tasks:
a) Explain why batch gradient descent is slow in large datasets.
Answer :

- Incorrect derivative of the activation function.
- Errors in applying the chain rule during backpropagation.

- Wrong sign in weight updates (adding instead of subtracting gradients).
- Incorrect learning rate implementation.
- Shape or indexing errors in matrix operations.
- Gradients not reset before each update.

b) Analyze the cause of fluctuations in SGD.

Answer :

Activation functions introduce non-linearity, allowing neural networks to learn complex patterns. During backpropagation, their derivatives determine how gradients flow through the network. Functions with very small derivatives (e.g., sigmoid at extreme values) can weaken gradient flow, while functions like ReLU help gradients pass more effectively.

c) Compare the error behavior of Mini-Batch Gradient Descent.

Answer :

- **Vanishing gradients:** Gradients become extremely small, causing early layers to learn very slowly or stop learning.
- **Exploding gradients:** Gradients become excessively large, leading to unstable weight updates and increasing training loss.
- Both problems reduce training efficiency and prevent the model from converging.

d) Recommend the best approach and justify with error trade-offs.

Answer :

- Use proper weight initialization (e.g., Xavier or He initialization).
- Normalize inputs using data normalization or batch normalization.
- Use ReLU or its variants instead of sigmoid/tanh in deep networks.
- Apply gradient clipping to prevent exploding gradients.
- Choose an appropriate learning rate or use adaptive optimizers such as Adam.
- Monitor training loss and gradients to detect issues early.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

Answer :

High-dimensional data contains many features, increasing model complexity. This makes it easier for the model to fit noise in the training data, leading to overfitting and poor generalization on unseen test data.

b) Identify signs of overfitting in this scenario.

Answer :

- Very high training accuracy but low test accuracy.
- Large gap between training and testing performance.
- Low training error but high validation/test error.
- Model performs poorly on new, unseen data.

c) Analyze why distance-based learning fails in high dimensions.

Answer :

In high-dimensional spaces, the distances between data points become very similar, making it difficult to distinguish between nearest and farthest neighbors. As a result, distance-based algorithms such as K-Nearest Neighbors (KNN) become less accurate and less effective.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answer :

- Apply **Principal Component Analysis (PCA)** or **Linear Discriminant Analysis (LDA)** to reduce the number of features.
- Use **feature selection** to remove irrelevant or redundant features.
- Apply **L1 (Lasso)** or **L2 (Ridge)** regularization to reduce overfitting.
- Increase the amount of training data or use cross-validation for better generalization.
- Use simpler models with fewer parameters when appropriate.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand